Hackathon 2026 – Organized as part of SEMICON India

KLA Problem Statement

AI-Based Restoration of Degraded Images for Semiconductor Inspection

1. Background

Inspection and imaging systems rarely capture perfectly clean images. Noise and loss of resolution can hide useful details and reduce the reliability of later computer-vision tasks. Image restoration attempts to recover the best possible clean image from a degraded observation.

For this challenge, KLA provides paired training examples: a clean ground-truth image and its degraded noisy, low-resolution version. Participants must learn a robust transformation that restores the degraded input.

SIMPLE EXPLANATION

Imagine receiving a small, grainy photograph. Your solution must remove the grain, recover missing detail and produce a larger, cleaner image that is as close as possible to the original.

2. Problem Description

Develop a reproducible AI-based image-restoration pipeline that:

- accepts degraded noisy, low-resolution images as input;
- handles the three specified degradation mechanisms: speckle noise, additive Gaussian noise and downsampling;
- produces restored images at the expected ground-truth resolution;
- generalizes to both familiar and unfamiliar image content; and
- runs efficiently as a complete inference pipeline on an NVIDIA GPU.

The three degradations may have been applied in any order. Your model does not need to identify that order explicitly; it may restore the image in one step or use a staged approach.

3. Objective

- Maximize restored-image quality without hallucinating or destroying real structure.
- Balance pixel fidelity, structural similarity and perceptual quality.

- Generalize beyond the image content seen during training.
- Optimize end-to-end throughput, not only the neural-network forward pass.
- Submit code and documentation that evaluators can run and reproduce without manual source-code edits.

4. Scope & Key Requirements

A. Dataset and image rules

Item	Confirmed requirement
Training data	Paired clean ground-truth (GT) and degraded noisy low-resolution (NoisyLR) images.
Hidden test data	Only degraded inputs will be released/provided for testing; KLA retains the clean targets for scoring.
Degradations	Speckle noise, additive Gaussian noise and downsampling only; their order is not disclosed.
Image range	GT values are normalized to [0,1]. NoisyLR values may extend slightly outside [0,1]; this is intentional.
Image sizes	Use the dimensions supplied in the official dataset. The sessions indicated evaluation images will be approximately 256×256 or 512×512.
Test distribution	Includes in-distribution and out-of-distribution image content. Noise mechanisms remain the same; sampled levels may vary within a similar range.
Scoring input	KLA will score the images exactly as saved by the submitted pipeline; apply any clipping or normalization inside your own solution.

B. Model development

- Any suitable restoration architecture may be used: CNN, transformer, algorithm-unrolling method, another published architecture or a justified custom/hybrid design.
- Open-source pretrained weights and public external datasets are allowed when their licences permit competition use.
- Disclose every external dataset/model with name, link, licence and paper or model/dataset card.
- You may create extra synthetic degraded pairs from the provided GT images.
- Choose and justify preprocessing, augmentation, architecture and losses. Frequency-domain methods are allowed but not mandatory.
- There is no fixed parameter-count limit, but unnecessarily large models may lose throughput.

C. Mandatory inference behaviour

- Provide a standalone inference script that accepts an input-directory argument and an output-directory argument.
- The script must load every degraded image, restore it and save each output image to the output directory.
- Preserve the required file naming and image format stated in the official dataset/evaluator instructions.
- Support NVIDIA GPU execution; batch processing is preferred when GPU memory permits.
- Include all model weights, configuration and dependencies required for execution.
- Do not require evaluators to edit source code, notebook cells or local paths.

RUNTIME DEFINITION

End-to-end runtime includes disk reading, preprocessing, CPU-to-GPU transfer, model execution, GPU-to-CPU transfer, post-processing and saving restored images.

D. Validation and reporting

- Create a validation split that is not used for training or model selection leakage.
- Report PSNR, SSIM and LPIPS; also include any additional metric used to select the final model.
- Compare at least one baseline with the final method.
- Show restored examples at full image resolution, including successful and failed cases.
- Report end-to-end runtime, batch size, hardware, software versions and timing method.
- Track experiments, random seeds, hyperparameters, checkpoints and final configuration.

5. Phase-wise Deliverables

Phase 1 - Solution submission

Mandatory item	What it must contain
1. Solution PPT/PPTX	Problem understanding, approach, model, losses, augmentation, experiments, PSNR/SSIM/LPIPS, runtime, examples, limitations, external resources and next steps.
2. GitHub repository link	Accessible repository containing the complete submission package and clear folder structure.
3. Inference script	Standalone Python script accepting input and output directories and writing restored images.
4. Training code	Reproducible script(s) used to produce the submitted checkpoint.
5. Model weights/config	Final checkpoint, architecture/configuration files and download instructions if permitted by the portal.

Mandatory item	What it must contain
6. README	Exact environment setup and training/inference commands, input/output contract and assumptions.
7. Dependencies	requirements.txt or equivalent environment specification with compatible versions.
8. Results/output samples	Metric summary, representative restored images and failure analysis.

Phase 1 registration and submission deadline: 16 August 2026. Follow the portal for the official cut-off time, naming convention and upload-size rules.

Later evaluation / shortlisting stage

Shortlisted submissions may be executed on KLA's hidden test data. The evaluation process will compare restored outputs with hidden ground truth and benchmark the full pipeline on a common NVIDIA H100 GPU. Do not retrain on hidden test inputs unless a later official instruction explicitly permits it.

Recommended repository structure

```
repository/
  README.md
  requirements.txt
  train.py
  inference.py
  configs/
  src/
  weights/
  results/
  solution_presentation.pptx
```

6. Evaluation Parameters

Evaluation axis	What evaluators will examine
Restoration quality	A fixed internal combination of PSNR, SSIM and LPIPS on hidden ground truth; both in-distribution and out-of-distribution image content.
End-to-end throughput	Total inference-pipeline time on a common NVIDIA H100, including image I/O and pre/post-processing.
Training & compute hygiene	Reproducibility, clean experiment design, environment specification, code quality, efficient data pipeline and standard ML/DL practices.

IMPORTANT

KLA confirmed that a fixed weighted combination is used but did not disclose the exact metric or axis weightages. No single target score or latency threshold has been prescribed.

7. Recommended Solution PPT Structure

1. Slide 1: Title, team and one-line solution.
2. Slide 2: Problem understanding and restoration task.
3. Slide 3: Dataset analysis and degradation observations.
4. Slide 4: End-to-end pipeline.
5. Slide 5: Preprocessing and augmentation.
6. Slide 6: Model architecture and design rationale.
7. Slide 7: Loss functions and training setup.
8. Slide 8: Experiment tracking and baseline comparison.
9. Slide 9: PSNR, SSIM and LPIPS results.
10. Slide 10: Runtime, batch size and optimization.
11. Slide 11: Visual results, failure cases and limitations.
12. Slide 12: Conclusion, external-resource disclosure and repository link.

8. Student Workflow

13. Download the official dataset and visually inspect GT/NoisyLR pairs and value ranges.
14. Build the data loader and a very small baseline model with a simple loss.
15. Overfit one or two pairs as a pipeline sanity check.
16. Create a clean train/validation split and record a baseline.
17. Test augmentations, models and losses one change at a time; track every experiment.
18. Inspect images as well as metrics so you can identify missed noise or lost detail.
19. Measure PSNR, SSIM, LPIPS and full-pipeline runtime on the chosen final checkpoint.
20. Run inference from a clean environment using only input/output directory arguments.
21. Package the PPT and GitHub repository, then verify every link and command before submitting.

9. Final Submission Checklist

☐ Mandatory solution PPT/PPTX is included.

☐ GitHub repository link is accessible.

☐ Only the three official degradation mechanisms are treated as benchmark requirements.

☐ NoisyLR values outside [0,1] are handled intentionally.

☐ Inference script accepts input and output directory arguments.

☐ Training script reproduces the submitted checkpoint.

☐ Model weights/configuration and environment specification are included.

☐ README commands run without manual source-code edits.

☐ PSNR, SSIM and LPIPS are reported.

☐ Both numerical metrics and restored-image examples are shown.

☐ End-to-end runtime, hardware, batch size and timing method are stated.

☐ At least one baseline and one failure case are included.

☐ External data/models include links and licence details.

☐ No confidential, unlicensed or inaccessible data is used.

☐ Submission has been dry-run in a clean environment.

10. Official Resources & Links

Use the following official resources before developing and submitting your solution. Open each link and confirm that your team can access it.

KLA detailed problem-statement PPT: Open link

Read this first for the sponsor's original technical explanation, dataset structure, evaluation focus and expected solution.

Official KLA dataset: Open link

Download the paired GT and NoisyLR training images. Preserve the folder structure and filenames unless later official instructions say otherwise.

Webinar 1 - Problem Statement Explanation: Open link

Recording of the 30 July 2026 session explaining the challenge, degradations, metrics, runtime expectations and submission requirements.

Webinar 2 - KLA Key Concepts & Q&A: Open link

Recording of the 7 August 2026 technical session covering practical guidance and participant questions.

Hackathon idea-submission template: Open link

Use this organizer-provided slide template for the mandatory Phase 1 solution presentation and follow the latest portal instructions for final PDF/PPT format.

Official hackathon landing page: Open link

Check this page for registration, updated deadlines, problem-statement notices, webinar resources and the latest submission instructions.

BEFORE SUBMISSION

Re-open every link, confirm that the dataset and templates are accessible, and use the latest portal notice if any instruction has changed.

11. Notes from the KLA Sessions

Problem Statement Explanation Webinar - 30 July 2026

- The task is paired image restoration from NoisyLR to clean GT using only speckle noise, additive Gaussian noise and downsampling as benchmark degradations.

- NoisyLR may contain values outside [0,1], while GT stays within [0,1].
- Public data and pretrained weights are allowed; evaluation covers quality, H100 throughput and reproducible training/compute hygiene.

Key Concepts & Q&A - 7 August 2026

- Hidden testing includes familiar and unfamiliar image content, but not unseen degradation mechanisms; PSNR, SSIM and LPIPS are combined internally.
- KLA does not clip or renormalize outputs; end-to-end timing includes image I/O, transfers, processing, model execution and saving.
- The final package requires inference code, reproducible training code, weights, dependencies and clear documentation; the organizer recommends a solution PPT.

12. References Shared by KLA

- Kumar, T. et al. (2024). Image Data Augmentation Approaches: A Comprehensive Survey and Future Directions. IEEE Access, 12.
- Zhai, L. et al. (2023). A Comprehensive Review of Deep Learning-Based Real-World Image Restoration. IEEE Access, 11, 21049-21067.
- Terven, J. et al. (2025). A Comprehensive Survey of Loss Functions and Metrics in Deep Learning. Artificial Intelligence Review, 58, 195.
- Monga, V. et al. (2021). Algorithm Unrolling: Interpretable, Efficient Deep Learning for Signal and Image Processing. IEEE Signal Processing Magazine, 38(2), 18-44.